

Generowanie i interpretacja reguł eksperckich dla klasyfikacji filmów Netflix z wykorzystaniem ML i LLM

Sprawozdanie z projektu

Stas Kochevenko, Diana Hunchak

2025/2026

1. Wprowadzenie

Celem projektu było zbudowanie interpretowalnego systemu ekspertowego, który klasyfikuje filmy dostępne w zbiorze Netflix jako *successful* albo *unsuccessful*, a następnie przedstawia uzasadnienie decyzji w postaci reguł IF-THEN. System łączy trzy mechanizmy: uczenie nadzorowane za pomocą drzewa decyzyjnego, symboliczną reprezentację wiedzy oraz model językowy LLM, który porządkuje i tłumaczy reguły na język zrozumiały dla użytkownika.

Środowiskiem uczenia jest zbiór danych o filmach Netflix, zawierający początkowo 16 000 rekordów i 18 kolumn. Dane opisują m.in. rok wydania, kraj i język produkcji, gatunki, popularność, liczbę głosów, średnią ocenę, budżet i przychód. Po odrzuceniu filmów bez ocen oraz produkcji z mniej niż 50 głosami pozostało 12 185 obserwacji. Filtr ogranicza wpływ przypadkowych ocen opartych na bardzo małej próbie.

W projekcie określenie *sukces* ma precyzyjne, techniczne znaczenie. Klasa *successful* oznacza film, którego *vote_average* jest nie mniejsze od mediany w oczyszczonym zbiorze, równej 6,40. Nie jest to bezpośrednia miara zysku, rentowności ani biznesowego sukcesu platformy Netflix. Średnia ocena służy wyłącznie do utworzenia etykiety uczącej i nie jest przekazywana do klasyfikatora jako cecha.

2. Koncepcja rozwiązania

2.1. Przygotowanie danych i reprezentacja stanu

Przetwarzanie danych obejmowało następujące operacje:

- usunięcie identyfikatorów i pól tekstowych (*show_id*, *title*, *description*) oraz cech o wysokiej kardynalności (*cast*, *director*);
- usunięcie pól technicznych *date_added* i *type*;
- usunięcie kolumny *duration*, ponieważ zawierała 100% braków;
- uzupełnienie braków w zmiennych kategoriowych etykietą *Unknown*;
- wyznaczenie głównego gatunku *main_genre* jako pierwszej pozycji z listy gatunków;
- usunięcie *vote_average* i *rating*, aby uniknąć wycieku informacji o etykiecie;
- kodowanie cech *country*, *language* i *main_genre* metodą One-Hot Encoding.

Stan analizowanego filmu jest zatem wektorem cech liczbowych i binarnych. Część liczbową obejmuje *release_year*, *popularity*, *vote_count*, *budget* i *revenue*; część binarna reprezentuje kraj, język i główny gatunek. Dane podzielono warstwowo na zbiór treningowy 80% oraz testowy 20%, używając *random_state=42*. Drzewa decyzyjne nie wymagają standaryzacji zmiennych, dlatego nie zastosowano skalowania.

2.2. Algorytm, reguły i miary jakości

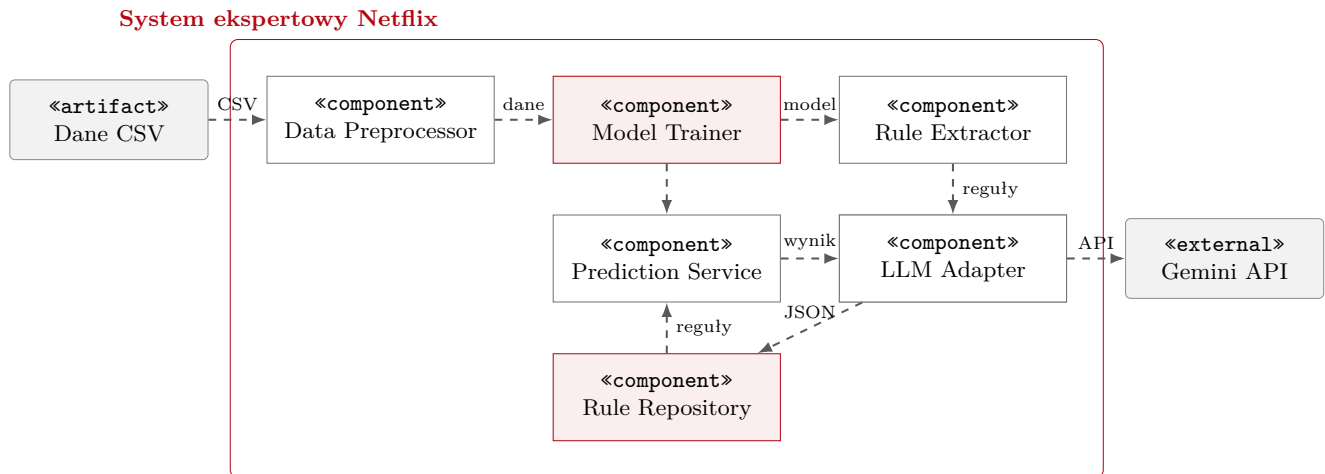
Klasyfikatorem jest *DecisionTreeClassifier* o maksymalnej głębokości 5 i minimalnej liczbie 20 obserwacji w liściu. Ograniczenia zmniejszają ryzyko przeuczenia oraz utrzymują liczbę reguł na poziomie możliwym do interpretacji. Każda ścieżka od korzenia do liścia jest zamieniana na regułę:

```
IF main_genre_Horror > 0.500
AND popularity > 73.279
AND vote_count > 878.000
THEN class = successful
```

Dla każdej reguły obliczane są dwie miary. *Support* oznacza liczbę rekordów treningowych spełniających wszystkie warunki. *Confidence* to udział poprawnie sklasyfikowanych rekordów wśród rekordów objętych regułą. Do dalszej analizy wybierane są reguły o wysokiej wiarygodności; prezentacja projektu przyjmuje próg *confidence* $\geq 0,75$.

LLM nie podejmuje właściwej decyzji klasyfikacyjnej. Otrzymuje reguły zapisane jako obiekty JSON wraz z klasą, *support* i *confidence*, a następnie: tłumaczy je na język naturalny, grupuje podobne wzorce, wskazuje redundancje, opisuje wyjątki oraz dodaje komentarze o granicach decyzji. Zastosowano model Gemini 2.5 Flash Lite. Odpowiedź jest parsowana i walidowana; przy błędzie system zachowuje poprzednią poprawną wersję reguły.

3. Architektura rozwiązania



Rysunek 1: UML Component Diagram systemu. LLM wspiera budowę bazy wiedzy, a predykcja pozostaje deterministyczna.

Implementacja została przygotowana w notebooku Jupyter. Biblioteka `pandas` odpowiada za przetwarzanie danych, `scikit-learn` za podział zbioru, uczenie i ewaluację drzewa, a `matplotlib` za wizualizację. Integrację z LLM zrealizowano przez pakiet `google-genai`. Reguły są serializowane do JSON, dzięki czemu można kontrolować wymagane pola i oddzielić generatywną interpretację od wykonania systemu ekspertowego.

Podczas predykcji dane pojedynczego filmu są kodowane do tego samego zestawu kolumn co dane treningowe. Model zwraca klasę i prawdopodobieństwo liścia, a moduł wyjaśniający odtwarza dokładną ścieżkę w drzewie. Demo nie wymaga ponownego wywołania LLM: korzysta z wytrenowanego klasyfikatora i zapisanej, zwalidowanej bazy wiedzy. Ogranicza to koszt, opóźnienie i niedeterministyczność odpowiedzi.

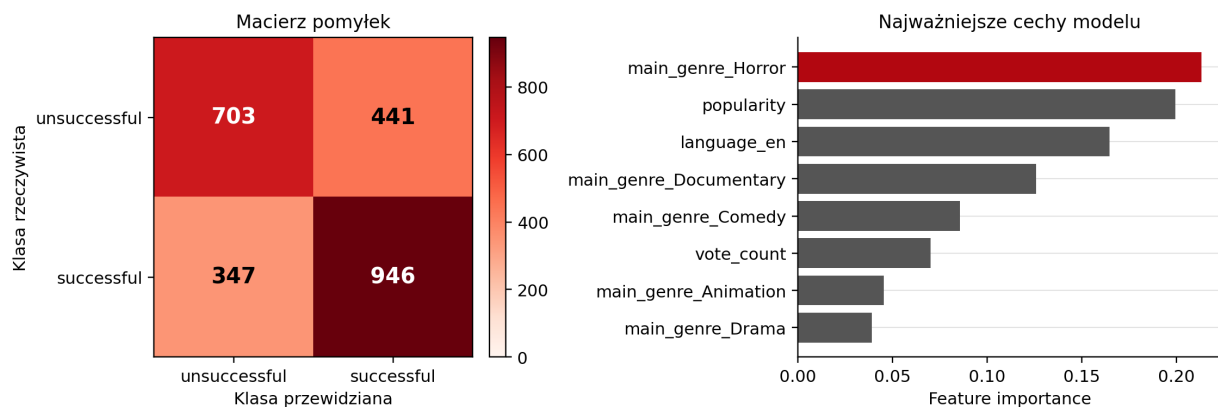
4. Wyniki eksperymentów i testów

4.1. Jakość klasyfikacji

Dla maksymalnej głębokości 5 i co najmniej 20 obserwacji w liściu model osiągnął `accuracy` 0,6767. Wyniki klasowe przedstawia tabela 1. Model lepiej rozpoznaje klasę `successful`: `recall` wynosi 0,73, podczas gdy dla klasy `unsuccessful` jest równy 0,61. Różnica nie wskazuje jednak na całkowite zdominowanie predykcji przez jedną klasę.

Tabela 1: Metryki na zbiorze testowym liczącym 2 437 filmów.

Klasa	Precision	Recall	F1-score	Liczba próbek
unsuccessful (0)	0,67	0,61	0,64	1 144
successful (1)	0,68	0,73	0,71	1 293
Średnia ważona	0,68	0,68	0,68	2 437

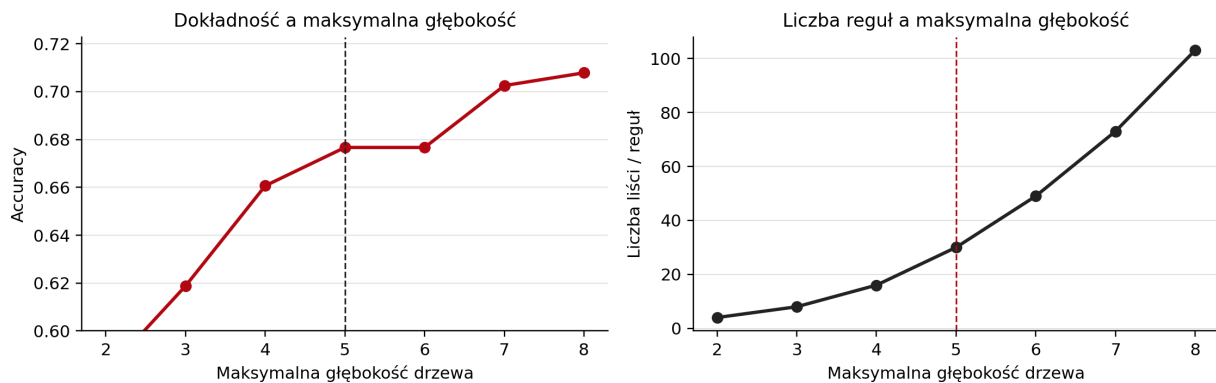


Rysunek 2: Macierz pomyłek oraz najważniejsze cechy drzewa o głębokości 5. Poprawnie sklasyfikowano 703 filmy klasy 0 i 946 filmów klasy 1.

Macierz pomyłek zawiera 441 przypadków klasy 0 błędnie uznanych za sukces oraz 347 przypadków klasy 1 błędnie uznanych za porażkę. Największy wpływ na decyzje mają binarna cecha gatunku **Horror** oraz **popularity**; istotne są również język angielski, inne gatunki i liczba głosów. Reguły wskazują m.in., że horrory o niskiej popularności częściej trafiają do klasy 0, natomiast dokumenty i animacje częściej trafiają do klasy 1. Są to zależności statystyczne w analizowanym zbiorze, a nie uniwersalne prawa branżowe.

4.2. Wpływ doświadczenia modelu i złożoności drzewa

Zgodnie z wymaganiem porównano skuteczność dla maksymalnej głębokości od 2 do 8. W tym zadaniu “wzrost doświadczenia” interpretujemy jako możliwość uczenia coraz bardziej szczegółowych podziałów. Największa poprawa występuje między głębokościami 2 i 4. Dalsze zwiększanie głębokości daje mniejsze korzyści dla accuracy, natomiast szybko zwiększa liczbę liści, czyli reguł **IF-THEN**.



Rysunek 3: Kompromis pomiędzy skutecznością klasyfikacji a interpretowalnością. Linia przerywana oznacza wybraną głębokość 5.

Wybrana głębokość 5 zapewnia czytelniejszą bazę wiedzy niż warianty głębsze, zachowując większość dostępnej skuteczności. Jest to istotne, ponieważ głównym celem projektu nie było wyłącznie maksymalizowanie accuracy, ale stworzenie systemu, którego decyzje można prześledzić i wyjaśnić.

4.3. Ocena reguł i działania LLM

Najlepsza reguła uzyskała confidence 1,000 przy support równym 23. Inne reguły osiągnęły m.in. 0,968 przy support 220 oraz 0,954 przy support 217. Wynik pokazuje konieczność wspólnej analizy obu miar: bardzo wysoka pewność przy małym support może oznaczać regułę precyzyjną, ale mało ogólną.

W zaktualizowanym etapie LLM analizuje 19 reguł wejściowych. Po usunięciu powtarzających się warunków i scaleniu reguł różniących się wyłącznie nieistotnym progiem pozostało 16 reguł. Wszystkie obiekty przeszły walidację wymaganych pól, a 15 z 16 otrzymało pole `conflict_note`. Ponieważ liście jednego drzewa definiują rozłączne obszary przestrzeni cech, nie są to bezpośrednie sprzeczności logiczne; komentarze opisują przede wszystkim wyjątki, anomalie i granice decyzji.

Test demonstracyjny dla filmu `Horror`, `popularity=80` i `vote_count=1500` dopasował regułę z progami `popularity > 73.279` i `vote_count > 878`. System zwrócił klasę `successful`, confidence 1,000 oraz support 23, jednocześnie prezentując użytkownikowi warunki prowadzące do decyzji.

5. Instrukcja uruchomienia

Projekt można uruchomić lokalnie lub w środowisku Google Colab:

1. umieścić notebook oraz plik CSV w tym samym katalogu i zainstalować wymagane biblioteki;
2. uruchomić kolejno komórki preprocessingu, uczenia modelu oraz ewaluacji;
3. dla etapu interpretacji reguł ustawić klucz Gemini API w zmiennej środowiskowej;
4. opcjonalnie zapisać wygenerowany plik JSON z uproszczonymi regułami.

Pliki źródłowe: `netflix_expert.ipynb`, `netflix_movies_detailed_up_to_2025.csv`.

Pliki wygenerowane: `tree_model.pkl`, `feature_columns.pkl`, `rules_final.json`.

6. Wnioski

Projekt potwierdził, że drzewo decyzyjne może jednocześnie pełnić rolę klasyfikatora i źródła symbolicznej wiedzy. Osiągnięta dokładność około 68% jest umiarkowana, ale model pozostaje przejrzysty: każda decyzja ma jednoznaczną ścieżkę i może zostać zapisana jako reguła ekspercka. LLM poprawia czytelność bazy wiedzy, lecz nie zastępuje modelu i nie powinien samodzielnie zmieniać znaczenia reguł bez walidacji struktury. Największą zaletą rozwiązania jest możliwość interpretacji decyzji modelu w postaci czytelnych reguł IF-THEN.

Najważniejsze ograniczenia to umowna definicja sukcesu na podstawie ocen, duża liczba kolumn po zakodowaniu krajów oraz możliwość słabej generalizacji reguł o małym support. Pole kraju zawiera czasem kilka państw zapisanych w jednym ciągu, dlatego One-Hot Encoding traktuje całe kombinacje jako odrębne kategorie.